

CS2023 - Programación III
Programación concurrente
Ejercicios de programación concurrente
Rubén Rivas Medina
Mayo 2025

1. Uso de hilos con archivos (std::thread)

Ejercicio 1

Leer un archivo **numeros.txt** con valores enteros entre 1 y 1000. Crea cuatro hilos que clasifiquen estos valores y los escriban en archivos separados: **pares.txt** (múltiplos de 2), **mult3.txt** (múltiplos de 3), **mult5.txt** (múltiplos de 5) y **otros.txt** (ninguna de las anteriores).

Ejercicio 2

A partir del archivo **numeros.txt**, lanza diez hilos que calculen de forma concurrente la suma de valores en rangos definidos: 1–10, 11–20, …, 91–100.

Ejercicio 3

Lee los archivos generados en el primer ejercicio y utiliza cuatro hilos para calcular la suma total de los valores de cada archivo (**pares.txt**, **mult3.txt**, **mult5.txt**, **otros.txt**).

2. Protección con mutex (std::mutex)

Ejercicio 4

Carga **numeros.txt** y divide su contenido en fragmentos para múltiples hilos. Cada hilo actualizará un **std::unordered_map<int, int>** para contar la frecuencia de cada número. Usa **std::mutex** para proteger el acceso concurrente al mapa.

Ejercicio 5

Considere la clase **Client** que representa a un cliente de un banco. Esta clase tiene como atributo principal al saldo (**balance**) del tipo **double**, además de un constructor por defecto y uno por parámetros que permita definir el saldo inicial. Se incluyen los métodos **deposit** y **withdraw** para actualizar el saldo, además de la sobrecarga del operador **<<** que permite imprimir el saldo.

1. Desarrollar un programa que genere un objeto del tipo **Client** y realice **nd** depósitos de valor **d** de forma concurrente, de igual modo realice **nr** retiros de valor **r** también en paralelo. Finalmente, utilizando el operador **<<**, imprimir el saldo final del cliente.
2. Considere realizar **m** objetos **Client** que representan a **m** clientes de un banco y realizar una simulación similar al ejercicio anterior para cada cliente. Finalmente, imprimir los valores de los saldos utilizando el operador **<<**.

Ejercicio 6

Diseña una clase **Logger** que permita a múltiples hilos escribir mensajes en un archivo compartido **log.txt** de forma segura. La clase debe:

- Usar un atributo interno **std::mutex** para proteger las operaciones de escritura.
- Proveer un método **log(std::string mensaje)** que escriba el mensaje seguido de un salto de línea.
- Abrir el archivo en modo **append** para mantener el historial de mensajes.

Nota: Asegúrate de que el **std::mutex** proteja todo el bloque de escritura para evitar condiciones de carrera.
Como extensión del ejercicio 1, implementa una clase **Logger** y úsala para registrar, desde cada hilo, cuántos elementos fueron procesados y en qué archivo se escribieron.

3. Coordinación con semáforo binario (std::binary_semaphore)

Ejercicio 7

Implementa un productor y un consumidor: el productor genera un número aleatorio y lo almacena en un buffer; el consumidor lo imprime. Sincroniza ambos usando un **std::binary_semaphore** para alternar el acceso.

Ejercicio 8

Dispones de tres ficheros de texto: **A.txt**, **B.txt** y **C.txt**, cada uno con varias líneas de números enteros. Crea tres hilos, cada uno asociado a uno de los ficheros. Cada hilo deberá:

1. Esperar su turno con un **std::binary_semaphore** para leer y procesar una línea del archivo.
2. Imprimir el número leído por pantalla.
3. Señalar al siguiente hilo (A→B→C→A…) liberando su semáforo.

De esta forma, la salida será siempre en el orden: línea de **A.txt**, línea de **B.txt**, línea de **C.txt** y de nuevo **A.txt**, hasta que se agoten todas las líneas de los ficheros.

Ejercicio 9

Simule un sistema en el que múltiples personas (representadas por hilos) desean ingresar a una sala que solo puede ser ocupada por una persona a la vez.
Implemente una función **ingresar_a_la_sala(std::string nombre)** que represente el intento de una persona por ingresar. Cuando un hilo accede a la sala, debe:

1. Esperar su turno utilizando un **std::binary_semaphore**.
2. Imprimir un mensaje como **[nombre] ha entrado a la sala**.
3. Permanecer dentro durante un tiempo simulado (por ejemplo, usando **sleep_for**).
4. Imprimir un mensaje como **[nombre] ha salido de la sala**.
5. Liberar el semáforo para permitir que otra persona entre.

Cree al menos 5 hilos con nombres diferentes (por ejemplo: **“Ana”**, **“Luis”**, **“Sofía”**, etc.), y asegúrese de que los mensajes se impriman sin superposición ni acceso simultáneo a la sala.